57. SMART CANTEEN ORDER AND STOCK MANAGEMENT

1. AIM

To write a Python program to manage inventory and billing for a cafeteria/canteen by checking menu item selection, validating order quantity against available stock, updating inventory, and printing total sales revenue.

2. PROBLEM STATEMENT

Automated point-of-sale systems in canteens need to prevent overselling of stock while generating immediate order bills and tracking daily revenue. Write a Python program that validates order items, checks stock availability, updates remaining inventory, and calculates total sales.

3. OBJECTIVES

• Initialize parallel lists for menu items, prices, and available stock.
• Display item menu and process multiple user orders.
• Validate item menu selection (1 to 3) and quantity constraints.
• Update remaining stock dynamically and sum up total order bill values.

4. ASSUMPTIONS & REQUIREMENTS

Assumptions: Menu contains 3 items: Idli (Rs 20), Dosa (Rs 40), Juice (Rs 25). Stock quantities are positive integers.
Requirements: Python 3.x Interpreter, PyCharm environment.

5. ALGORITHM

Step 1: Set items = ['Idli', 'Dosa', 'Juice'], prices = [20, 40, 25], stock = [30, 20, 15], sales = 0.
Step 2: Display menu options.
Step 3: Read number of orders n.
Step 4: Loop n times: Input choice, qty. Validate choice and stock. Compute bill, deduct stock, update sales, and display bill.
Step 5: Display total sales and remaining stock vector.
Step 6: Stop program.

6. PROGRAM FLOWCHART

7. PYTHON PROGRAM (Smart Canteen Order and Stock.py)

```
items = ["Idli", "Dosa", "Juice"]
prices = [20, 40, 25]
stock = [30, 20, 15]
sales = 0

for i in range(3):
    print(i + 1, items[i], "Rs.", prices[i])

n = int(input("Enter number of orders: "))

for i in range(n):
    choice = int(input("Choose item (1-3): "))
    qty = int(input("Enter quantity: "))

    if choice < 1 or choice > 3:
        print("Invalid item")
    elif qty <= 0:
        print("Invalid quantity")
    elif qty > stock[choice - 1]:
        print("Not enough stock")
    else:
        bill = prices[choice - 1] * qty
        stock[choice - 1] -= qty
        sales += bill
        print("Bill: Rs.", bill)

print("Total sales: Rs.", sales)
print("Remaining stock:", stock)
```

8. EXPLANATION OF THE CODE

• stock[choice - 1] -= qty: Updates remaining inventory dynamically upon purchase.
• sales += bill: Accumulates total sales revenue across order transactions.

9. TRACING TABLE

Order #	Choice	Qty	Item Cost	Bill	Updated Stock (Dosa)	Total Sales
Order 1	2 (Dosa)	1	Rs. 40	Rs. 40	19	Rs. 40
Order 2	2 (Dosa)	1	Rs. 40	Rs. 40	18	Rs. 80

10. TEST CASES

Test Case	Input	Expected Result	Actual Result
TC 1 (Normal Orders)	Orders: 2, Choice 2 Qty 1, Choice 2 Qty 1	Bill: Rs. 40, Bill: Rs. 40, Total sales: Rs. 80, Stock: [30, 18, 15]	Bill: Rs. 40, Bill: Rs. 40, Total sales: Rs. 80, Stock: [30, 18, 15]

11. EXPECTED OUTPUT

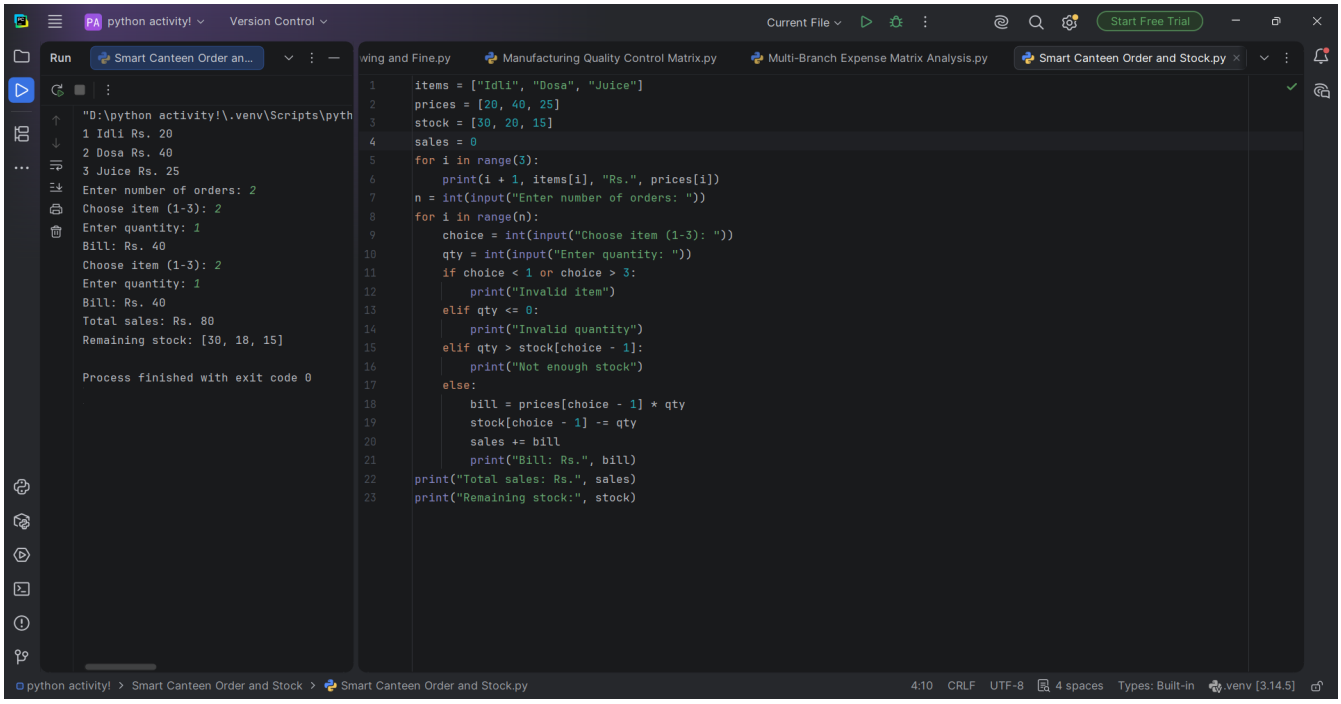
```
1 Idli Rs. 20
2 Dosa Rs. 40
3 Juice Rs. 25
Enter number of orders: 2
Choose item (1-3): 2
Enter quantity: 1
Bill: Rs. 40
Choose item (1-3): 2
Enter quantity: 1
Bill: Rs. 40
Total sales: Rs. 80
Remaining stock: [30, 18, 15]
```

12. ACTUAL OUTPUT

```
1 Idli Rs. 20
2 Dosa Rs. 40
3 Juice Rs. 25
Enter number of orders: 2
Choose item (1-3): 2
Enter quantity: 1
Bill: Rs. 40
Choose item (1-3): 2
Enter quantity: 1
Bill: Rs. 40
Total sales: Rs. 80
Remaining stock: [30, 18, 15]
```

13. PROGRAM EXECUTION SCREENSHOT

Actual Input and Output - PyCharm Run Screenshot



14. RESULT/CONCLUSION

The smart canteen order management script was executed successfully, managing order billing and updating stock inventory accurately.